

Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?

Authors: Sewon Min, Xinxu Lyu, Ari Holtzman, Mikel Artetxe, Michael Lewis

Year: 2022 · Venue: · Difficulty: ●●○○○ (Intermediate)

[10.18653/v1/2022.emnlp-main.759](https://arxiv.org/abs/10.18653/v1/2022.emnlp-main.759)

TL;DR

This paper shows that in-context learning (getting a language model to do a task from a few examples in the prompt) often does not depend much on the examples having correct labels; instead, much of the gain comes from showing the model the possible answer words, the kind of input text it will see, and the overall prompt format.

Prerequisites

- Language model (a model that predicts the next word or token from previous text)
- In-context learning (doing a new task by putting a few examples in the prompt instead of updating model weights)
- Classification (choosing one label such as positive, negative, or neutral for an input)
- Multiple-choice prompting (asking the model to pick from a fixed set of answer options)
- Zero-shot vs few-shot (zero-shot means no examples in the prompt; few-shot means a small number of examples are included)
- Prompt format (the exact text pattern used to present examples and ask for an answer)
- Label space (the set of possible output labels, like positive, negative, neutral)
- Input distribution (the kind of text the model is seeing, such as movie reviews versus news headlines)
- Meta-training (extra training designed to make a model better at learning from prompts)

How to Read This

5 minutes: TL;DR + The Big Idea + Key Contributions.

20 minutes: Add The Problem + Where This Fits.

The Big Idea

The paper asks a very basic question that people had mostly been hand-waving around in 2022: when a language model gets better from few-shot prompting, what exactly is it learning from those demonstrations? The common story was simple. You show the model a few input-output pairs, and it infers the rule. If the examples say review → sentiment label, then the model supposedly learns that mapping on the fly. This paper says: not so fast. They run a blunt but very revealing test. They replace the correct labels in the demonstrations with random labels. Performance often barely drops.

That is the headline, and it is genuinely surprising. If the model were really learning the mapping from

the examples in the obvious way, randomizing the labels should wreck performance. But it does not. Across many models, including GPT-3, the hit is small on classification and multiple-choice tasks. So the demonstrations are helping, but not mainly by teaching the model which input goes with which label. That is the paper's central punchline.

What is helping, then? The authors break demonstrations into parts, almost like taking apart a machine to see which gears matter. They find three big ingredients: the demonstrations tell the model what the possible labels are, what kind of inputs are coming, and what output format to use. In plain English, the examples act less like a tiny training set and more like a very strong hint. They say, "We are doing sentiment now; the answers look like Positive/Negative/Neutral; the inputs look like finance headlines; respond in this pattern." That turns out to be enough to recover much of the gain.

Why should you care? Because this changes how we think about in-context learning. It pushes us away from the romantic idea that the model is rapidly learning a new task at test time from a handful of examples. Instead, the model may mostly be locating a task it already knows from pretraining and then formatting its answer correctly. That is a big conceptual shift. It also matters for practice: if labels do not need to be correct to get much of the benefit, then prompt design is less about tiny supervised learning in the prompt and more about exposing the right answer set, input style, and structure.

The Problem

The specific problem is understanding why few-shot prompting beats zero-shot prompting. We knew empirically that adding demonstrations often helps. But we did not know what information inside those demonstrations was doing the work. Is it the true mapping from input to label? Is it just the answer words? Is it the style of the inputs? Is it the formatting? Without that breakdown, people were treating demonstrations like a black box.

This is hard because a prompt mixes several signals at once. Suppose you show the model:

$\text{Input}_1 \rightarrow \text{Positive}, \quad \text{Input}_2 \rightarrow \text{Negative}, \quad \text{Test Input} \rightarrow ?$

That prompt contains at least four things: the task format, the candidate labels, examples of the input domain, and the actual pairings between inputs and labels. If performance improves, you cannot tell which of these caused it unless you carefully intervene on one piece at a time. Most earlier work focused on making prompting work better, not on isolating these causal pieces.

The authors' key move is almost embarrassingly simple: scramble the labels while keeping the rest of the prompt intact. If performance stays high, then the true input-label correspondence was not doing much. They then go further and test partial prompts that preserve only some ingredients. This is the right experimental logic. It is like checking whether a student solved a problem by understanding the lesson or by recognizing the answer format and topic. The paper's answer is pretty clear: current language models often lean heavily on prior knowledge from pretraining, and demonstrations mostly help them retrieve and package that knowledge rather than learn a fresh rule on the spot.

Key Contributions

- They show that replacing gold labels with random labels in demonstrations often causes only a small drop in performance across 12 models, including GPT-3. This matters because it directly challenges the standard story that in-context learning mainly comes from learning the input-label mapping from the prompt.

- They identify three major useful parts of demonstrations: the label space, the input distribution, and the overall format. This gives a much sharper picture of what demonstrations are actually doing.
- They show that models can keep most of the few-shot gain even when given only partial information, such as just the inputs or just the label set, as long as the format is right. That is a strong sign that prompt structure itself is a major source of performance.
- They compare different model families, including meta-trained models (models specially trained to learn from prompts), and find that meta-training can amplify reliance on shallow prompt cues like format. That is both interesting and a little worrying, because it suggests some apparent in-context learning may be more brittle than it looks.
- They argue for a reframing of in-context learning: often the model is not learning a new task at test time in the strict supervised sense, but instead using demonstrations to locate a task it already knows from pretraining. This is the paper's biggest conceptual contribution.

Where This Fits

This paper sits in the early wave of work trying to understand, not just improve, prompting. Before this, most of the energy was on making few-shot prompting work better: better prompt wording, better example selection, better scoring methods, and later instruction tuning. That work was useful, but it mostly treated demonstrations as obviously informative training examples. Min et al. challenge that assumption directly.

The paper also connects to a broader debate about what large language models are doing at inference time. One line of work suggested that in-context learning might look like Bayesian inference (updating beliefs from evidence) or some kind of fast learning inside the forward pass. Another line argued that models mostly use knowledge already stored during pretraining and that prompts just help them access it. This paper lands firmly closer to the second camp. It does not say demonstrations are useless. Far from it. It says their main role is more like task specification than task teaching. That idea has aged well and influenced how people think about prompting, instruction following, and the limits of test-time adaptation.

How It Works

First, the simple version: the paper does not invent a new model. It takes standard large language models (language models; systems that predict the next token) and asks a very sharp question: what part of a prompt with demonstrations is actually helping? In normal in-context learning, you give the model k example input-output pairs, then a new input, and ask for the output. The common story is that the model learns the mapping from inputs to labels from those examples. This paper tests that story by breaking the demonstrations in controlled ways and seeing what still works.

The base setup is straightforward. A demonstration prompt looks like $(x_1, y_1), \dots, (x_k, y_k)$ followed by a test input x . Here x_i is an example input, like a sentence, and y_i is its label, like Positive or Negative. They evaluate 12 model configurations total: 6 decoder-only dense language models (autoregressive models; they generate text left-to-right), each used with two inference styles: direct and channel. Model sizes range from 774M parameters up to 175B for GPT-3. They also include MetaICL, which starts from GPT-2 Large and is meta-trained (trained across many tasks to get good at few-shot prompting) with an in-context learning objective. The key architectural choice is not a new network block. It is the comparison between prompting styles and between raw pretrained models versus a model explicitly

trained to use demonstrations.

Now the full picture: they separate four possible sources of signal inside demonstrations. These are: (1) the input-label mapping (whether each example has the correct label), (2) the input distribution (whether the example texts look like the task data), (3) the label space (whether the labels come from the real set, like Positive/Negative, versus random words), and (4) the format (whether the prompt is structured as input-label pairs at all). Then they build ablations that hold some of these fixed and destroy others. For example, “random labels” keeps the same inputs and same pair format, but breaks the true mapping. “OOD + random labels” uses out-of-distribution text (text from the wrong distribution) and random labels, so it keeps only the rough pair format. “Random English words” keeps the inputs and pair format but replaces labels with unrelated words. “No labels” and “labels only” remove the pair structure entirely. This is the clever bit: instead of asking whether demonstrations help, they ask what exactly inside the demonstrations is doing the helping.

Inference is done in the standard scoring style for prompted language models. In the direct method, the model scores or generates the label given the demonstrations and the test input, so it is estimating something like $p_\theta(y \mid x, D)$ where D is the prompt. In the channel method, it flips the direction and scores the input conditioned on a candidate label and the demonstrations, effectively using something like $p_\theta(x \mid y, D)$ and then choosing the best label. This matters because direct models must generate the label tokens themselves, so they care more about the label words. Channel models condition on labels instead, so they often care more about whether the prompt format and input text look familiar. There is no new loss introduced in this paper for GPT-2, GPT-J, fairseq, or GPT-3 at test time; the only training-specific piece is MetaICL, which was already trained with a multitask in-context objective before evaluation.

The punchline of the method is brutally simple. They vary k , the number of demonstrations, usually looking at values like $k = 4$ and $k \geq 8$. They compare gold labels, random labels, no demonstrations, and the format/input/label-space ablations across classification and multiple-choice tasks. Then they read off which pieces matter. The answer is not what most people expected. Correct labels matter surprisingly little. In-distribution inputs matter a lot. The input-label pair format matters a lot. The label space matters mainly for direct models. And meta-training seems to make models lean even harder on these simpler cues instead of truly learning the mapping inside the prompt.

Wrong Labels Still Help

This is the headline result. If you replace correct demonstration labels with random wrong ones, performance barely drops in many settings. That means the model is not mainly learning the task by reading the example label assignments. Something else in the prompt is carrying most of the value.

The Prompt Is a Format Cue

The input-label pair structure itself is doing real work. It is like giving the model a filled-in form and then one blank line at the end. The model often succeeds because it recognizes the pattern and completes it, not because it inferred the true rule from the examples.

In-Distribution Text Matters More Than Truth

Using example inputs from the right data distribution helps a lot, even when their labels are random. That is impressive and a little unsettling. It suggests the model benefits from seeing the right kind of text nearby, almost like warming up on the task’s style before answering.

Meta-Training Teaches Shortcuts

MetaICL is especially insensitive to wrong labels and especially sensitive to prompt format. The authors' read is blunt: training a model to do in-context learning may teach it to exploit the easiest cues first. In other words, it becomes better at using prompts, but not necessarily in the way people hoped.

Abstract

Large language models (LMs) are able to in-context learn—perform a new task via inference alone by conditioning on a few input-label pairs (demonstrations) and making predictions for new inputs. However, there has been little understanding of *how* the model learns and *which* aspects of the demonstrations contribute to end task performance. In this paper, we show that ground truth demonstrations are in fact not required—randomly replacing labels in the demonstrations barely hurts performance on a range of classification and multi-choice tasks, consistently over 12 different models including GPT-3. Instead, we find that other aspects of the demonstrations are the key drivers of end task performance, including the fact that they provide a few examples of (1) the label space, (2) the distribution of the input text, and (3) the overall format of the sequence. Together, our analysis provides a new way of understanding



Figure 1: Results in classification (top) and multi-choice tasks (bottom), using three LMs with varying size. Reported on six datasets on which GPT-3 is evaluated; the channel method is used. See Section 4 for the full results. In-context learning performance drops only marginally when labels in the demonstrations are replaced by random labels.

This is the teaser figure, and it makes the paper's point fast: across model sizes, using random labels in demonstrations barely hurts compared with gold labels. The thing to notice is not just the small gap. It is that the gap stays small even as models get bigger.

(3) meta-training with an in-context learning objective (Min et al., 2021b) magnifies these effects—the models almost exclusively exploit simpler aspects of the demonstrations like the format rather than the input-label mapping.

In summary, our analysis provides a new way of understanding the role of the demonstrations in in-context learning. We empirically show that the model (1) counter-intuitively does not rely on the ground truth input-label mapping provided in the

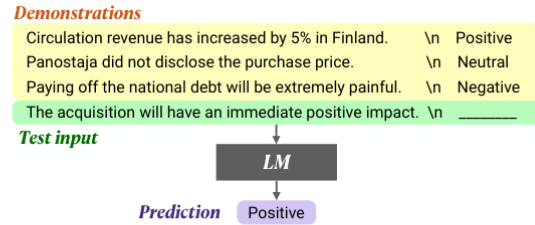


Figure 2: An overview of in-context learning. The demonstrations consist of k input-label pairs from the training data ($k = 3$ in the figure).

Figure 2: An overview of in-context learning. The demonstrations consist of k input-label pairs from the training data ($k = 3$ in the figure).

The Math

Let $D = \{(x_i, y_i)\}_{i=1}^k$ be the set of k demonstrations, where x_i is an input text and y_i is its label. Let x be the test input, and let C be the candidate label set.

For direct prompting (predict the label from the input), the model scores each candidate label $y \in C$ by

$$\hat{y}_{\text{direct}} = \arg \max_{y \in C} \log p_{\theta}(y \mid D, x). \quad (1)$$

In plain English: the model looks at the whole prompt, then asks, “which label string is most likely to come next?” This is the usual few-shot classification setup people have in mind.

If the label y is a sequence of tokens $y = (y^{(1)}, \dots, y^{(m)})$, then the score is tokenized as

$$\log p_{\theta}(y \mid D, x) = \sum_{t=1}^m \log p_{\theta}(y^{(t)} \mid D, x, y^{(<t)}). \quad (2)$$

This just says the model generates the label one token at a time. That matters because direct models are sensitive to the exact label words, like whether the class is written as "positive" versus "good".

For channel prompting (predict the input from the label), the model flips the direction and scores

$$\hat{y}_{\text{channel}} = \arg \max_{y \in C} \log p_{\theta}(x \mid D_y), \quad (3)$$

where D_y is the prompt formed by the demonstrations together with the candidate label y placed in the test template.

In plain English: instead of asking “how likely is this label?”, it asks “if this were the label, how well would the model explain the test input?” This often reduces weird biases from label token frequency.

Expanding the channel score over input tokens $x = (x^{(1)}, \dots, x^{(n)})$ gives

$$\log p_{\theta}(x \mid D_y) = \sum_{t=1}^n \log p_{\theta}(x^{(t)} \mid D_y, x^{(<t)}). \quad (4)$$

This means the model is evaluating how naturally the test input fits after seeing the candidate label in context. That is why channel models can care less about the exact label words and more about prompt structure.

The paper’s real contribution is not a new objective, but a decomposition of what information the demonstrations contain. We can write the demonstration prompt as carrying four factors:

$$D \equiv (F, L, I, M), \quad (5)$$

where F is format (input-label pair structure), L is label space (the set or style of labels used), I is input distribution (whether the example texts come from the same distribution as the task), and M is input-label mapping correctness.

In plain English: the prompt is not one thing. It is a bundle of cues. The whole paper is about pulling those cues apart and checking which ones the model actually uses.

The standard gold-demonstration setting keeps all four factors:

$$D_{\text{gold}} = (F = 1, L = 1, I = 1, M = 1). \quad (6)$$

This is the normal few-shot prompt: right format, right labels, right kind of inputs, and correct pairings. If performance is high here, that alone does not tell you which of the four factors mattered.

The random-label condition breaks only the mapping:

$$D_{\text{rand-label}} = (F = 1, L = 1, I = 1, M = 0). \quad (7)$$

This is the key stress test. If performance stays high, then the model was not relying much on the true correspondence between each example input and its label.

The out-of-distribution-input plus random-label condition keeps only format and label space:

$$D_{\text{OOD+rand}} = (F = 1, L = 1, I = 0, M = 0). \quad (8)$$

Here the examples are still written as pairs, and the labels still look like the task labels, but the example texts come from the wrong source. If performance drops a lot, that tells you the model was using the style or distribution of the example inputs.

The random-English-word condition keeps format and input distribution, but removes the real label space:

$$D_{\text{rand-word}} = (F = 1, L = 0, I = 1, M = 0). \quad (9)$$

This asks whether the actual class names matter. For direct models, they often do. For channel models, much less.

The no-label and labels-only conditions remove the pair format:

$$D_{\text{no-labels}} = (F = 0, L = 0, I = 1, M = 0), \quad (10)$$

$$D_{\text{labels-only}} = (F = 0, L = 1, I = 0, M = 0). \quad (11)$$

These are the cleanest tests of format. If pair structure disappears and performance collapses, then the model was using the prompt as a structural template, not just as a bag of useful words.

To compare conditions, the paper effectively studies performance differences such as

$$\Delta_M = \text{Perf}(D_{\text{gold}}) - \text{Perf}(D_{\text{rand-label}}), \quad (12)$$

$$\Delta_I = \text{Perf}(D_{\text{rand-label}}) - \text{Perf}(D_{\text{OOD+rand}}), \quad (13)$$

$$\Delta_L = \text{Perf}(D_{\text{rand-label}}) - \text{Perf}(D_{\text{rand-word}}), \quad (14)$$

$$\Delta_F \approx \text{Perf}(\text{paired prompt}) - \text{Perf}(\text{unpaired prompt}). \quad (15)$$

These are not training losses. They are diagnostic gaps. Each one isolates how much one factor helps by comparing two prompt variants that differ in only that factor.

One especially useful summary statistic in the paper is the fraction of in-context learning gain retained under a corrupted prompt. Let P_{none} be performance with no demonstrations, P_{gold} with gold demonstrations, and P_{abl} with some ablation. Then

$$\text{RetainedGain} = \frac{P_{\text{abl}} - P_{\text{none}}}{P_{\text{gold}} - P_{\text{none}}}. \quad (16)$$

This tells you how much of the benefit survives after you break part of the prompt. For example, the paper reports that always using incorrect labels still preserves about 92%, 100%, and 97% of the improvement for MetaICL classification, MetaICL multi-choice, and GPT-J multi-choice, respectively.

Finally, MetaICL differs because it was trained with an in-context learning objective over many tasks. A generic form of that training objective is

$$\mathcal{L}_{\text{meta}}(\theta) = -\mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{(x,y) \in \tau_{\text{query}}} \log p_{\theta}(y \mid \tau_{\text{support}}, x) \right], \quad (17)$$

where τ is a task, τ_{support} is the demonstration set, and τ_{query} is the held-out example to predict.

In plain English: MetaICL is explicitly trained to solve new tasks from demonstrations. The paper’s striking claim is that this training may teach the model to exploit the easiest prompt cues, like format, even more aggressively than ordinary language-model pretraining does.

Does It Work?

The setup is broad enough to matter. They evaluate on 26 datasets total: 16 classification datasets and 10 multiple-choice datasets for most models, with GPT-3 evaluated on a smaller subset of 3 classification and 3 multiple-choice datasets. Task types include sentiment analysis, paraphrase detection, natural language inference, hate speech detection, question answering, and sentence completion. The models span from GPT-2 at 774M parameters up to GPT-3 at 175B, plus fairseq 6.7B, fairseq 13B, GPT-J, and MetaICL. Metrics are macro-F1 for classification and accuracy for multiple-choice. That is a sensible setup: broad tasks, broad scales, and both direct and channel prompting.

The headline result is genuinely surprising and holds across a lot of settings. In Figure 3, demonstrations with random labels are very close to demonstrations with gold labels for both classification and multiple-choice. The paper gives the sharpest summary in retained gains: always using incorrect labels preserves about 92% of the gain for MetaICL classification, 100% for MetaICL multiple-choice, and 97% for GPT-J multiple-choice. Even GPT-J classification, where wrong labels hurt more, still does much better with wrong-label demonstrations than with no demonstrations. That is impressive because it directly attacks the standard story that in-context learning works mainly by inferring the input-to-label rule from examples.

The ablations in Section 5 are where the paper really earns its keep. Changing the input distribution to out-of-distribution text causes a large absolute drop of about 3 to 16 points for Channel MetaICL, Direct GPT-J, and Channel GPT-J, in both classification and multiple-choice. In Direct GPT-J multiple-choice, the OOD variant is even worse than no demonstrations. So the model strongly cares whether the example inputs look like the task data. The label space test is more nuanced: replacing labels with random English words hurts direct models by about 5 to 16 absolute points, but barely affects channel models, often only 0 to 2 points. That split is exactly what you would expect if direct models need to

generate label tokens while channel models mostly use labels as conditioning context. Then the format test lands the final punch: if you remove the pair structure, performance is usually close to or worse than no demonstrations. In other words, the prompt format is not decoration. It is one of the main engines.

Now the honest caveats. First, the paper is about classification and multiple-choice tasks only. It does not test open-ended generation, code, long-form reasoning, or tool use. So do not overgeneralize. Second, the paper reports many trends as averages across datasets, which is useful, but it can hide task-specific failures. Third, the GPT-3 evaluation is on only 6 datasets total, much smaller than the rest. Fourth, they show that manual templates do not always beat minimal templates, which is interesting, but they do not deeply analyze why some templates help and others do not. Still, the core result is robust enough that these gaps do not kill it. The paper convincingly shows that a lot of what we call in-context learning is really prompt conditioning on format, label words, and in-distribution text, not clean task induction from labeled examples.

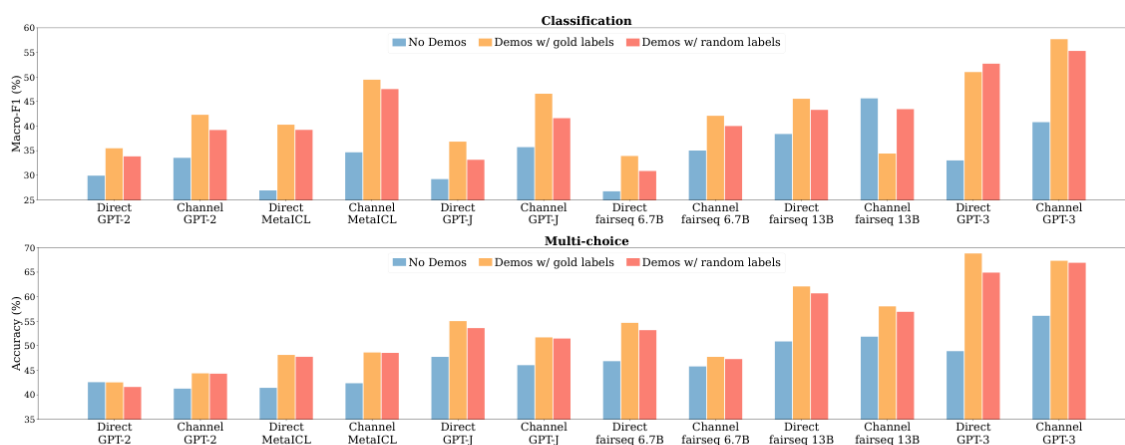


Figure 3: Results when using no-demonstrations, demonstrations with gold labels, and demonstrations with random labels in classification (top) and multi-choice tasks (bottom). The first eight models are evaluated on 16 classification and 10 multi-choice datasets, and the last four models are evaluated on 3 classification and 3 multi-choice datasets. See Figure 11 for numbers comparable across all models. **Model performance with random labels is very close to performance with gold labels** (more discussion in Section 4.1).

This is the main broad result across 12 model configurations. No demonstrations are clearly worse, but gold-label and random-label demonstrations are often very close. That is the figure that should make you stop and rethink what demonstrations are doing.

Demonstrations w/ random labels are formed with random labels, instead of gold labels from the labeled data. Each x_i ($1 \leq i \leq k$) is paired with $\tilde{y}_i$ that is randomly sampled at uniform from $\mathcal{C}$. A concatenation of $(x_1, \tilde{y}_1) \dots (x_k, \tilde{y}_k)$ is then used to make a prediction via $\operatorname{argmax}_{y \in \mathcal{C}} P(y | x_1, \tilde{y}_1 \dots x_k, \tilde{y}_k, x)$.

Results are reported in Figure 3. First, using the demonstrations with gold labels significantly improves the performance over no demonstrations,⁵ as it has been consistently found in much of prior

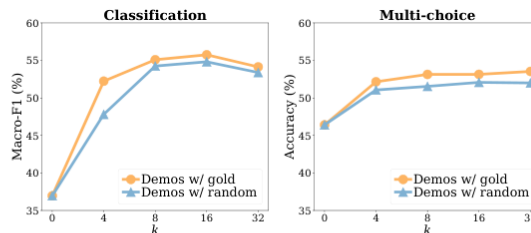


Figure 5: Ablations on varying numbers of examples in the demonstrations (k). Models that are the best under 13B in each task category (Channel MetaICL and Direct GPT-J, respectively) are used.

This figure varies the number of demonstrations k . The striking part is that performance improves a lot from no demos to a small number like $k=4$, but then saturates quickly around $k \geq 8$, and the gold-vs-random gap stays tiny. That is very unlike ordinary supervised learning, where more labeled examples usually help a lot.

demonstrations, e.g., preserving 92%, 100% and 97% of improvements from using the demonstrations with MetaICL in classification, MetaICL in multi-choice, and GPT-J in multi-choice, respectively. In contrast, GPT-J in classification sees relatively significant performance drop with more incorrect labels, e.g., nearly 10% drop in performance when always using incorrect labels. Still, always using incorrect labels is significantly better than no demonstrations.

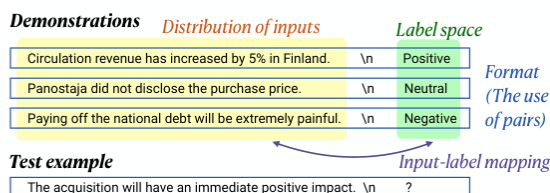


Figure 7: Four different aspects in the demonstrations: the input-label mapping, the distribution of the input text, the label space, and the use of input-label pairing as the format of the demonstrations.

This is the conceptual map of the whole paper. It breaks demonstrations into four parts: mapping, input distribution, label space, and format. Once you see the problem this way, the later ablations feel clean and almost obvious.

Blind Spots & Gotchas

Blind Spots & Gotchas

- Implementation gotcha: direct and channel scoring are easy to mix up. If you score $p_{\theta}(y | x, D)$ when you meant $p_{\theta}(x | y, D)$, your numbers will look plausible but your conclusions about label-space effects will be wrong.
- Implementation gotcha: for multi-token labels, you must sum token log-probabilities consistently across candidates. If you compare raw probabilities or forget length effects, short labels get an unfair advantage and the ablation results become garbage.
- Implementation gotcha: the ablations only isolate one factor if the prompt template is held fixed. If your OOD condition also changes formatting, spacing, separators, or label wording, you are no longer measuring input distribution alone.
- Paper blind spot: the study is mostly about classification and multiple-choice. It does not tell you whether the same story holds for chain-of-thought prompting, code completion, retrieval-augmented generation, or long-context reasoning.
- Paper blind spot: random labels are still drawn from the task label set in one key ablation. That means the model still sees the right answer vocabulary. For some applications, that is a big clue, not a minor detail.

So What?

This paper matters because it punctures a lazy story the field was telling itself. People often said in-context learning works because the model reads a few labeled examples and infers the task rule on the fly. This paper shows that is only part of the story, and often not the main part. A lot of the gain comes from simpler things: seeing in-distribution text, seeing the right label words, and seeing the familiar input-output format. That is a big conceptual correction. It changes how you should design prompts, how you should interpret few-shot gains, and how you should evaluate whether a model is really learning from examples.

In practice, this pushes you toward better prompt engineering and better benchmarks. If you want strong performance, spend effort on format and task-matching context, not just on finding perfectly

labeled demonstrations. If you want to test real task induction, you need harder controls, because standard few-shot prompting can look smart even when the labels are nonsense. In quantitative finance, this is directly useful. For forecasting and risk, you can build prompts with in-distribution examples from earnings calls, filings, analyst notes, or market news, and the formatting may matter as much as exact labels. For portfolio optimization and execution, if you use language models to classify event sentiment or detect regime shifts from text, this paper says: keep the prompt structure stable, keep examples from the same market domain, and do not assume the model is truly learning your label semantics from a handful of examples.

A concrete extension: rerun this decomposition for modern instruction-tuned and reasoning-tuned models on financial NLP tasks. For example, test whether a model classifying 8-K filings or earnings-call snippets still performs well when demonstration labels are randomized but the examples remain in-domain. Then compare downstream effects on a trading signal, like next-day abnormal return prediction or volatility forecasting. My guess is you would find the same pattern, maybe even stronger. Closing thought: this paper is valuable because it is not impressed by surface success. It asks what is really doing the work, and the answer is both humbling and useful.